BLAND2GRAND

Maker's Guide

The Smart, Autonomous Carousel Spice Dispenser

Version 1.0

Contents

1 Introduction

This Maker's Guide is written for someone who has the printed/purchased parts to build one in front of them and wants to assemble it and get it running. It covers two supported controller boards:

Arduino UNO Q
The primary, currently supported build. The Flask backend, dispense logic, and MCU sketch all run on a single board, launched through Arduino App Lab. Start here.

Arduino UNO R4 WiFi
The original build. The backend and frontend run on a separate laptop, communicating with the Arduino over a mobile hotspot.

Follow **Section 2** to physically assemble the machine, then **Section 3** to install and launch the software for whichever board you have.

At a Glance

Build Time	Skill Level	Boards Supported
A weekend for assembly, an afternoon for software	Intermediate — soldering, 3D printing, basic CLI	UNO Q (recommended), UNO R4 WiFi

2 Assembly

Section In Progress

This section will walk through the full mechanical and electrical assembly of Bland2Grand: printing and preparing parts, assembling the carousel and cycloidal gearbox, mounting the auger modules, wiring the motors, drivers, load cell, and controller board, and closing up the enclosure. Content to be added.

2.1 Parts and Tools Needed

Placeholder — bill of materials and tools list.

2.2 Printing the Parts

Placeholder — print settings, orientation, and material notes.

2.3 Mechanical Assembly

Placeholder — carousel, gearbox, auger modules, tower frame.

2.4 Electrical Wiring

Placeholder — motors, drivers, load cell, controller board wiring diagram.

2.5 Final Checks Before Power-On

Placeholder — pre-power checklist.

3 Code and Software Setup

All source code for Bland2Grand is hosted on GitHub: github.com/ElliottStarosta/Bland2Grand. The repository contains separate, self-contained builds for each supported board. Set up the one that matches your hardware.

3.1 Arduino UNO Q Setup (Recommended)

The UNO Q build runs the whole system (MCU sketch, Flask backend, REST API, SSE stream, and the built React frontend) as a single App Lab app on the board itself. There is no separate laptop backend/frontend to run.

- **Phase 1: Download the Project**

1. Navigate to the Bland2Grand GitHub repository.

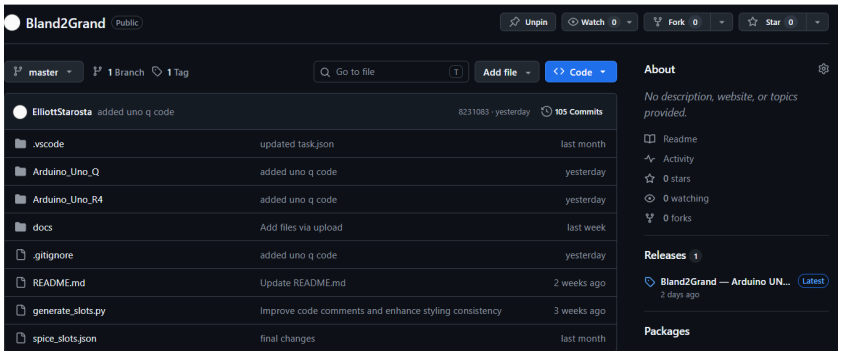

Figure 1: The Bland2Grand GitHub repository page.

2. Locate and click on the latest release.

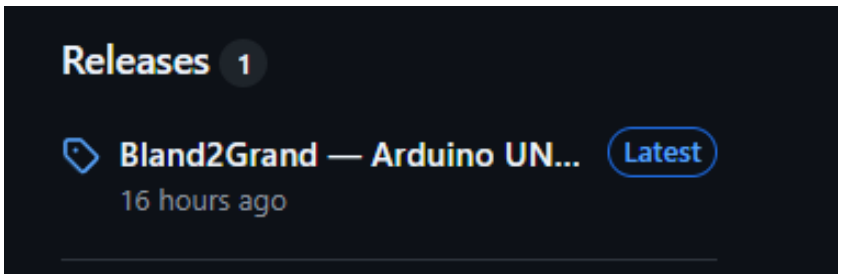

Figure 2: Latest release on the Releases page.

3. Scroll to the **Assets** section at the bottom of the release.

Figure 3: Assets section at the bottom of a release.

4. Download the zip file: `Bland2Grand_Arduino_Uno_Q.zip`.

Figure 4: Downloading `Bland2Grand_Arduino_Uno_Q.zip` from the Assets list.

- **Phase 2: Deploy to Arduino / App Lab**

5. Open **App Lab**.

6. Select your board.

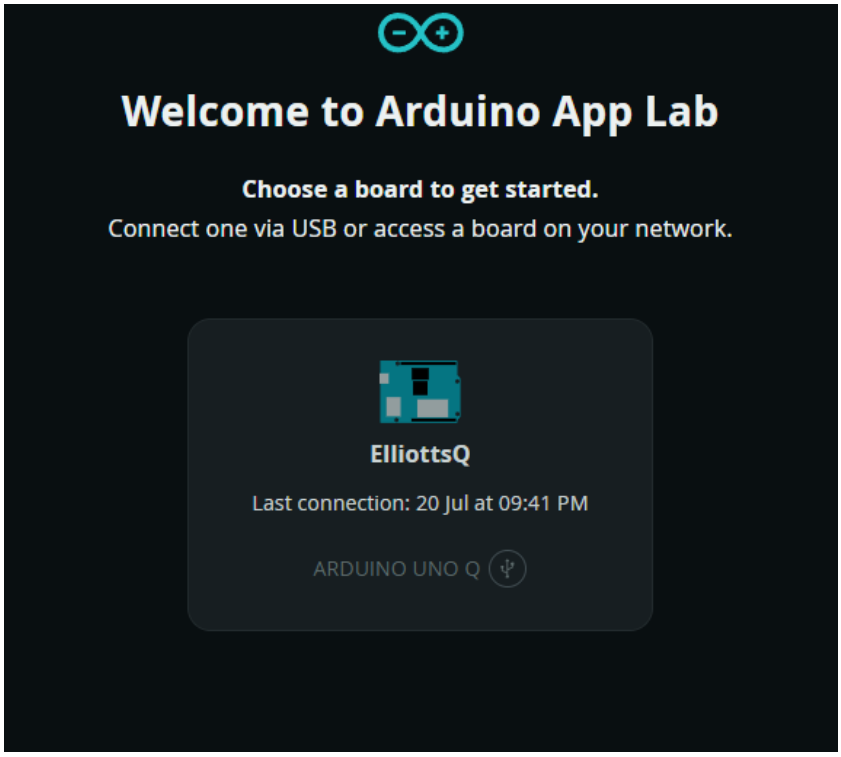

Figure 5: Selecting the connected UNO Q board in App Lab.

7. Click **Create new app** in the top right-hand corner.

Figure 6: The "Create new app" button in App Lab.

8. Click **Import App** and select the `.zip` file you downloaded in Phase 1.

Phase 3: Configure API Credentials

The AI-assisted custom blend feature calls out to OpenRouter. An API key is required for this feature to work (the rest of the app — search, dispensing, calibration — works without it).

9. Go to the OpenRouter keys page.

10. Click **New Key**.

Figure 7: The "New Key" button on the OpenRouter keys page.

11 Fill out the required information.

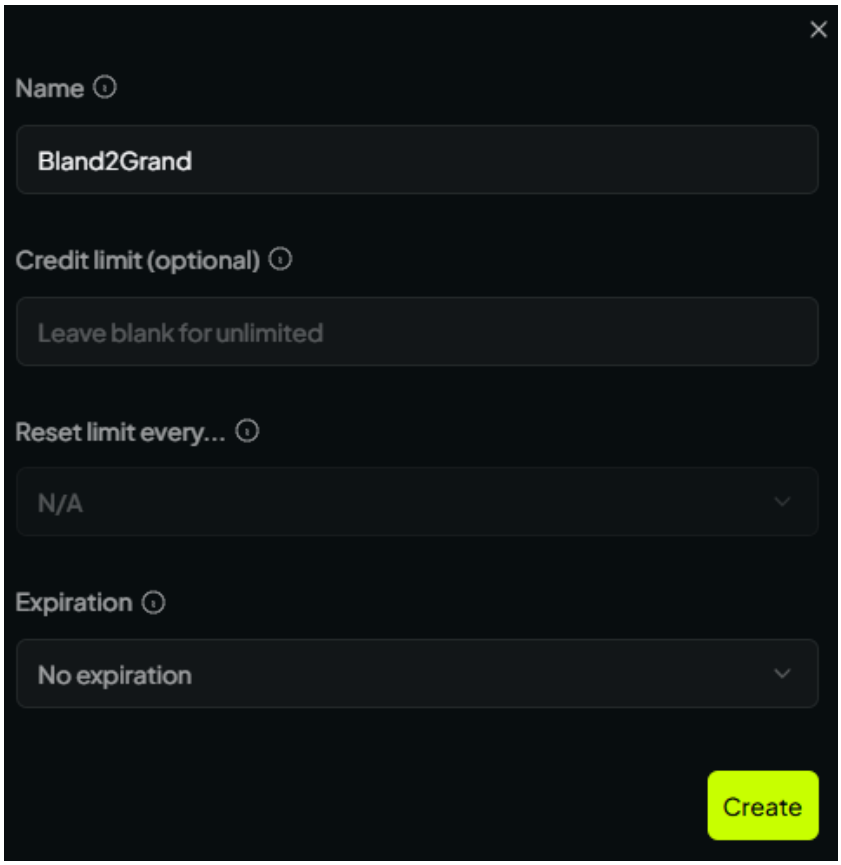

Figure 8: OpenRouter new key creation form.

12 Copy the key shown on the confirmation screen right away — OpenRouter will not show it to you again.

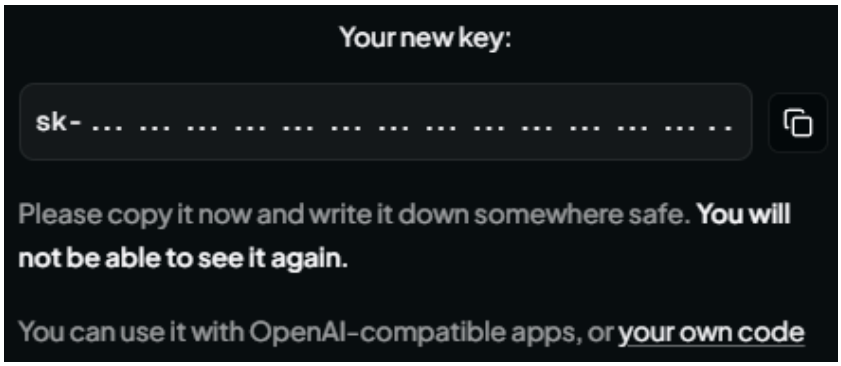

Figure 9: The one-time key reveal screen after creating a key.

13 Update the relevant environment variables for the app, most importantly `OPENROUTER_API_KEY`.

```
OPENROUTER_API_KEY=YOUR_KEY_HERE
AI_MODEL=nvidia/nemotron-3-super-120b-a12b:free
DATABASE_PATH=/app/python/bland2grand.db
MOCK_BRIDGE=false
FLASK_PORT=5000
FRONTEND_DIST=./dist
```

Figure 10: Example environment configuration for the UNO Q app.

Keep your API key private

Treat your OpenRouter key like a password: don't commit it to a public repository, share it in screenshots, or paste it into chat tools. If a key is ever exposed, revoke it from the OpenRouter dashboard and generate a new one.

- **Phase 4: Run the Application**

13 Set the application to run on startup by clicking the down arrow at the top left-hand side of the screen, next to "Bland2Grand".

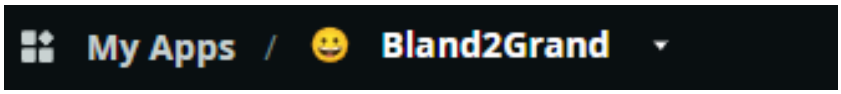

Figure 11: The startup dropdown next to the app name in App Lab.

14 Toggle the **Run at startup** option to enabled.

Figure 12: Enabling the "Run at startup" toggle.

15 Click the **Run** button in the top right-hand corner of the screen. App Lab uploads the MCU sketch and starts the Flask app on the board's Linux side.

16 Access the web server by opening the board's address on port **5000** (the port declared in `app.yaml`) in your browser.

17 You are now ready to use Bland2Grand.

Why enable Run at startup

With **Run at startup** enabled, the UNO Q automatically relaunches the Bland2Grand app any time it's powered on or rebooted, so you don't need App Lab open to bring the machine back up after a power cycle.

No separate frontend server

Unlike the UNO R4 build below, the UNO Q serves the built React frontend directly from the same Flask app that handles the API, so there is nothing else to start or forward — one board, one port, one app.

3.2 Arduino UNO R4 WiFi Setup

The UNO R4 build splits the system across three pieces: firmware on the Arduino, a Flask backend, and a React frontend, both of the latter run on a laptop and communicate with the Arduino over WiFi. This section assumes the hardware is already assembled and wired per **Section 2**, and that the `Bland2Grand_Arduino`, `bland2grand-backend`, and `bland2grand-frontend` folders (and the provided `.vscode/tasks.json`) are on the laptop that will run the software.

Hardware Power-On

1. Place the machine on a flat, level surface — the load cell platform must stay level to within 2° for accurate readings.
2. Connect the 24 V power supply to the wall.
3. Confirm each of the 8 spice containers is loaded and seated.
4. Place an empty bowl on the scale platform. The scale tares before each dispense, so the exact bowl weight does not matter.
5. Make sure the host laptop is powered on and ready to start its mobile hotspot (see below) — the Arduino and the phone running the web app both need to join it.

Launching the Software

All software is launched from VS Code using the provided `tasks.json`. Open the project workspace in VS Code, then open the Command Palette with **Ctrl+Shift+P** and select **Tasks: Run Task**.

Figure 13: VS Code Command Palette showing "Tasks: Run Task" selected.

Two compound tasks are available:

Task Name	What it does
Bland2Grand: Full Stack	Flashes firmware to the Arduino, opens the serial monitor, starts the Flask backend, and starts the Vite frontend — all in parallel. Use this for a full bring-up from scratch.
Bland2Grand: Backend + Frontend	Starts only the Flask backend and the Vite frontend. Use this when the Arduino is already flashed and running. This is the task to run for a demo.

What Full Stack does automatically

The **Arduino: Flash + Serial** subtask does three things before flashing: it starts the Windows mobile hotspot (so the Arduino has a network to connect to), kills any open serial monitor that would block the port, then runs `pio run --target upload` followed by `pio device monitor --baud 9600`. No manual hotspot setup or port management is needed.

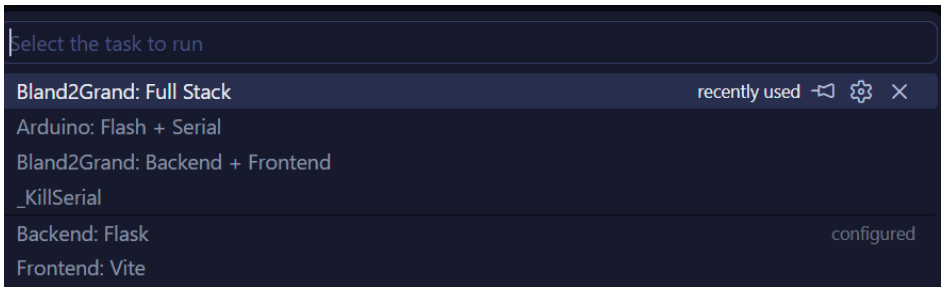

Figure 14: VS Code task picker showing the available Bland2Grand tasks.

Once the **Backend + Frontend** task is running, the web app is accessible from any device on the same network at:

`http://{laptop_ip}:5173`

The laptop's IP address is printed in the Vite terminal panel when it starts. On the Arduino's serial monitor you will also see the IP it received from the hotspot, which confirms the two devices are on the same subnet.

First-Time Backend Setup

If this is the first time running the backend on this laptop, set it up once before using the VS Code tasks:

```
cd bland2grand-backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python seed_recipes.py
```

Create a `.env` file in `bland2grand-backend/`:

```
ARDUINO_URL=ARDUINO_URL=http://192.168.x.x
OPENROUTER_API_KEY=YOUR_API_KEY_GOES_HERE
AI_MODEL=anthropic/claude-3-haiku
DATABASE_PATH=bland2grand.db
FLASK_PORT=5000
MOCK_ARDUINO=true
```

```
cd bland2grand-frontend
npm install
```

Open `Bland2Grand_Arduino` in PlatformIO once to confirm it builds before relying on the VS Code task to flash it.

(!) Set `MOCK_ARDUINO=false` for a real run

`MOCK_ARDUINO=true` simulates dispensing in software with no hardware attached — useful for testing the app without the machine. Set it to `false` once the Arduino is flashed and wired, so the backend talks to the real hardware.

Using the App

Once the web app is open on a phone, the user interacts with it the same way regardless of which hardware board is running underneath. The process begins by searching for a dish or creating a custom spice blend, selecting a recommended result, and adjusting the serving size based on the desired quantity. After confirming the blend, the user taps **Dispense**, which starts the automated dispensing process. A live progress screen provides real-time feedback as each spice is measured and dispensed, allowing the user to monitor the system's status. Once all required spices have been successfully delivered, the interface transitions to the **Complete** screen, indicating that the blend is ready to use.